Intermediate (chunking method changes the answer) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Same idea, as two functions: `chunk_by_chars()` and `chunk_by_paragraph()`, doing two different jobs on the same text.

`chunk_by_chars` treats the whole document as one long string and cuts it every `N` characters, with no idea where a sentence or even a word ends — it can slice straight through the middle of a word.

`chunk_by_paragraph` treats a blank line as the natural place to cut, because that's where the person who wrote the document already grouped their own ideas.

The exact pieces:

- Fixed-size slicing: `for i in range(0, len(text), chunk_size): chunks.append(text[i:i+chunk_size])`.
- Paragraph splitting: `text.split("\n\n")`, then `.strip()` each piece and drop any that are empty.
- Reuse `get_embedding` and `cosine_similarity` from `embedding_similarity` unchanged — you're applying the same math to more chunks, not learning new math.

Both `chunk_by_chars` and `chunk_by_paragraph`, as written so far, can fail in ways that only show up on a document that isn't as tidy as your test document.

`chunk_by_paragraph` assumes the document actually *has* blank-line breaks. Hand it one giant, single-paragraph wall of text — a scraped web page, a PDF that lost its formatting on the way in — and `text.split("\n\n")` returns exactly 1 "chunk": the entire document, unsplit. That's not a

crash, which is what makes it dangerous — it silently defeats the entire point of chunking, and you won't notice until a search for something near the end of that giant chunk comes back oddly.

`chunk_by_chars` has a different, subtler problem: even where it "works," any fact sitting near a `chunk_size` boundary gets cut clean in half between two *non-overlapping* chunks, and neither chunk alone contains the whole fact — this is exactly the failure the `chunk_boundary_split` exercise builds on purpose. The real fix isn't a bigger `chunk_size` (that just moves the problem, it doesn't remove it) — it's giving neighboring chunks a little **overlap**, so text near a boundary shows up whole in at least one chunk instead of split across two.

The extra pieces:

- A `chunk_by_chars` that steps forward by `chunk_size - overlap` instead of `chunk_size`, so each new chunk starts a little *before* where the last one ended.
- A fallback inside `chunk_by_paragraph`: if splitting on `"\n\n"` produces only 1 piece (or the piece is still enormous), fall back to `chunk_by_chars` instead of silently returning one giant "chunk."
- The chunker as a parameter: a function is a value, so `best_chunk_for(chunk_fn, text, question_vector)` can call whichever chunker it's given. Its type hint is `Callable[[str], list[str]]` (from `typing`) — "takes a `str`, returns a `list[str]`". Keep the chunkers in a `dict` of name → function and loop over it.

Sketch the overlapping version of `chunk_by_chars` yourself before checking Hint 2.

Hint 2 — The plan, and almost the whole thing

```
define:
    DOCUMENT = "... a few paragraphs, one topic ..."
    QUESTION = "... a question whose answer lives in one paragraph ..."

define:
    def find_best_chunk(chunks, question_vector) -> tuple[str, float]:
        embed every chunk, score it against question_vector, keep the best

main:
    char_chunks = chunk_by_chars(DOCUMENT)
    paragraph_chunks = chunk_by_paragraph(DOCUMENT)
    question_vector = get_embedding(QUESTION)

    for each method's chunk list:
        best_chunk, best_score = find_best_chunk(chunks, question_vector)

    print both methods' top chunk and score, side by side
    compare which one actually contains the full answer
```

```
# chunking_practice.py
def find_best_chunk(
    chunks: list[str], question_vector: list[float],
) -> tuple[str, float]:
    best_chunk = ""
    best_score = -1.0
    for chunk in chunks:
        chunk_vector = get_embedding(chunk)
        score = cosine_similarity(question_vector, chunk_vector)
        if score > best_score:
            best_score = score
            best_chunk = chunk
    return best_chunk, best_score
```

Write `DOCUMENT`, `QUESTION`, and the `main()` that calls everything, then compare both against the Solution.

Once that's working, add the overlap and fallback pieces:

```
function chunk_by_chars_with_overlap(text, chunk_size, overlap):
    chunks = []
    i = 0
    while i < len(text):
        chunks.append(text[i : i + chunk_size])
        i = i + (chunk_size - overlap)     # step forward LESS than a chunk_size
    return chunks

function chunk_by_paragraph_safe(text):
    pieces = chunk_by_paragraph(text)
    # blank-line split found nothing useful
    if len(pieces) <= 1:
        return chunk_by_chars_with_overlap(text, chunk_size=200, overlap=40)
    return pieces
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# chunking_practice.py
def chunk_by_chars_with_overlap(
    text: str, chunk_size: int = 200, overlap: int = 40,
) -> list[str]:
    chunks: list[str] = []
    i = 0
    step = chunk_size - overlap
    while i < len(text):
        chunks.append(text[i:i + chunk_size])
```

```
        # your turn: move i forward by `step`, not by `chunk_size` --
        # that's what makes each new chunk start inside the last one
        ...
    return chunks

def chunk_by_paragraph_safe(text: str) -> list[str]:
    pieces = chunk_by_paragraph(text)
    # your turn: if pieces has 1 item or fewer, the document had no real
    # paragraph breaks -- fall back to chunk_by_chars_with_overlap instead
    ...
```

Then pass the chunker in as a value:

```
from typing import Callable

def best_chunk_for(
    chunk_fn: Callable[[str], list[str]],
    text: str,
    question_vector: list[float],
) -> tuple[str, float]:
    # your turn: call chunk_fn on text, then reuse find_best_chunk
    ...
```

Fill in the overlap step, the fallback check and `best_chunk_for` yourself, then compare all of your finished versions against the Solution.

Full solution: Show me the solution